ESTRUCTURAS DE DATOS · MTRO EDSON G ESTRADA LÓPEZ

Torres de Hanoi

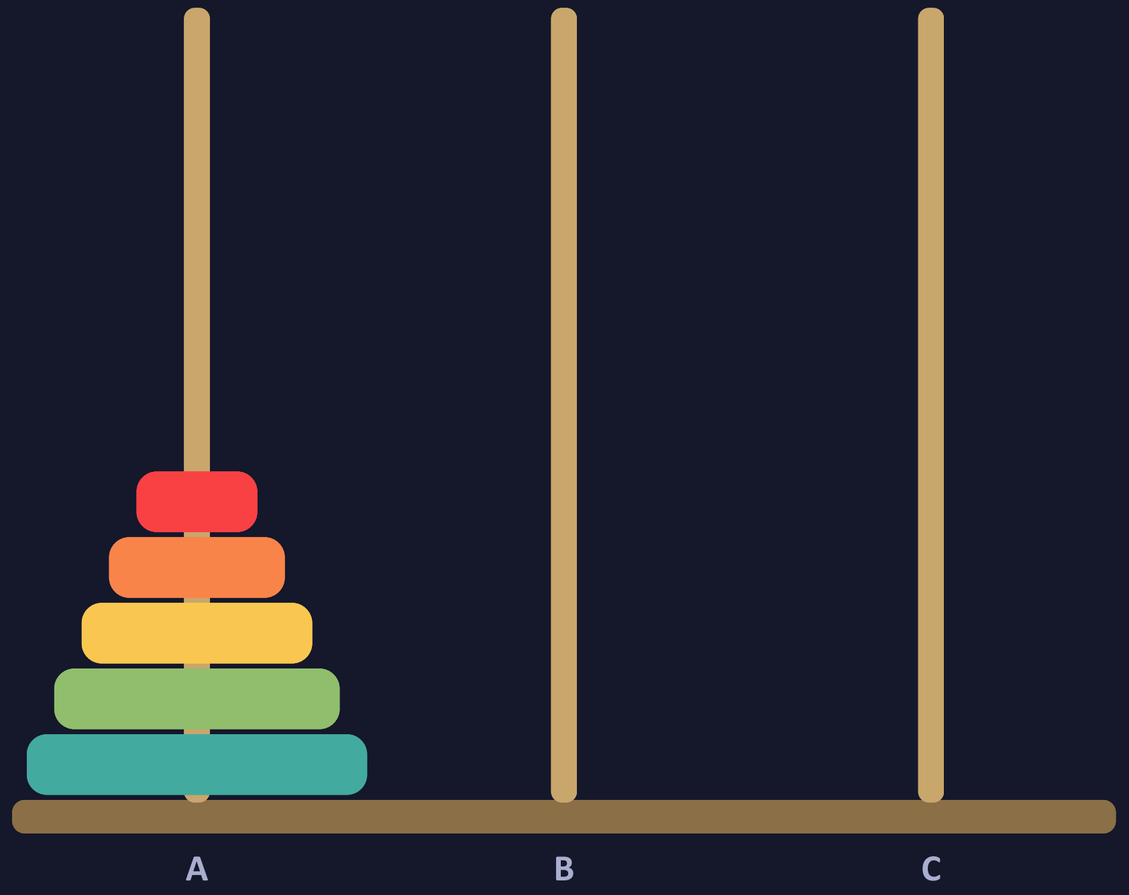
Universidad Modelo. Escuela de Ingeniería

Ingeniería en Desarrollo de Tecnologías y Software

Litzy García Richaud

Aymerik Aili Ocampo González

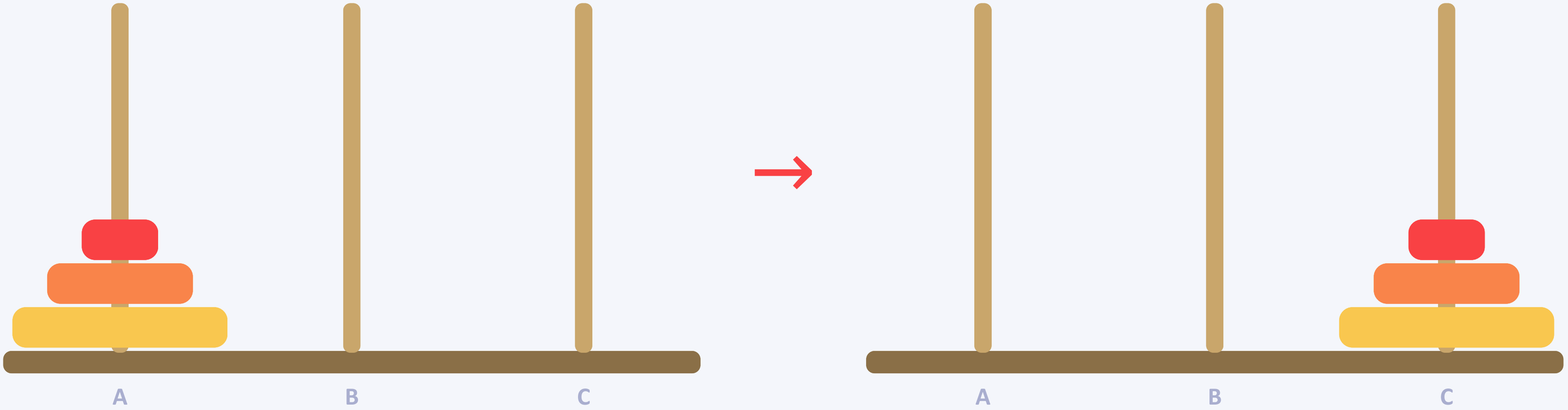
Octavio Augusto Novelo Martínez



Definición

Rompecabezas matemático de Édouard Lucas (1883)

El juego consiste en pasar todos los discos de la varilla ocupada (es decir la que posee la torre) a una de las otras varillas vacantes



Reglas del juego:

1. Sólo se mueve un disco a la vez
2. Un disco de mayor tamaño no puede descansar sobre uno más pequeño
3. Solo se puede mover el disco de arriba

Cuántos movimientos

Cada disco extra duplica el trabajo: **$2^n - 1$**

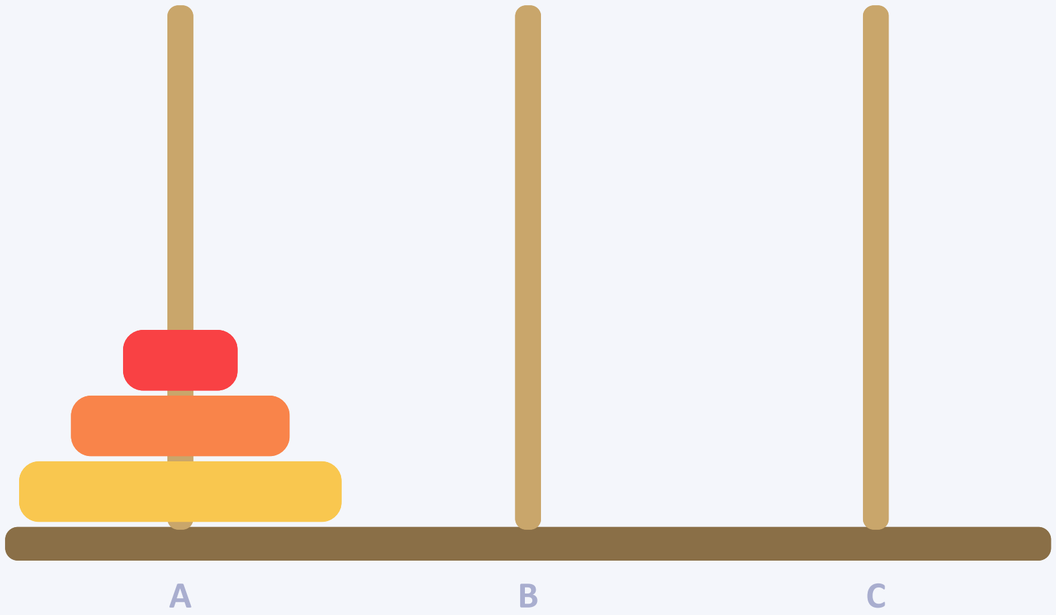
3 discos	5 discos	10 discos	20 discos	64 discos
7	**31**	**1,023**	**1,048,575**	**$\approx 1.8 \times 10^{19}$**
movimientos	movimientos	movimientos	movimientos	movimientos

Con 64 discos y un movimiento por segundo, la leyenda de la torre de Brahma tardaría unos 585 mil millones de años en terminar.

Ejemplo con 3 discos

Los 7 movimientos de A (origen) a C (destino), con B como auxiliar

Inicio

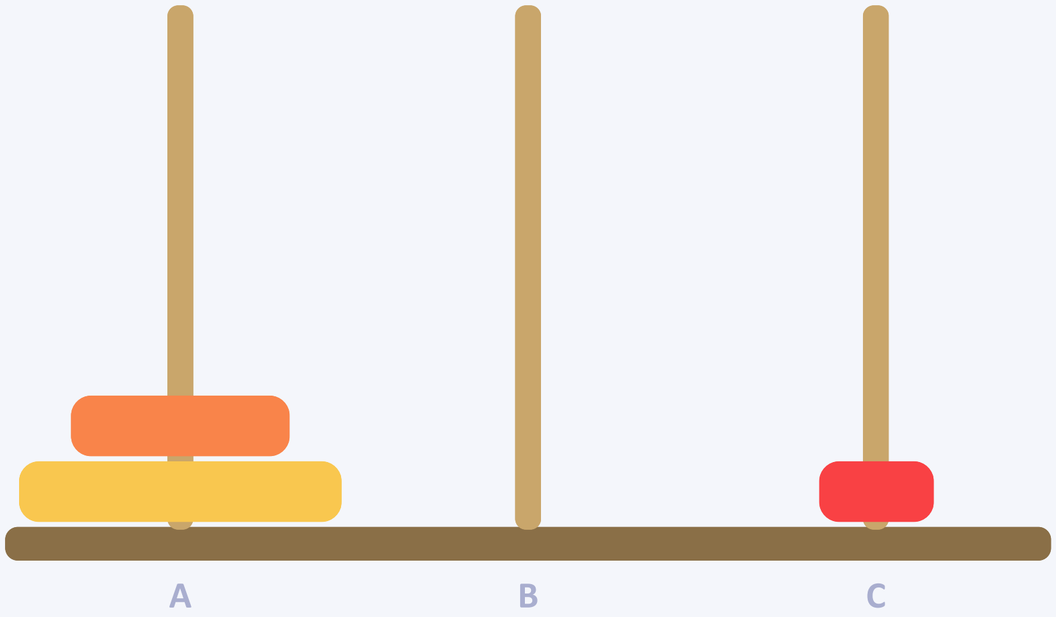


1 **A→C**

El disco 1 se mueve en los pasos 1, 3, 5 y 7, siempre en el ciclo A → C → B → A.

Ejemplo con 3 discos

Los 7 movimientos de A (origen) a C (destino), con B como auxiliar

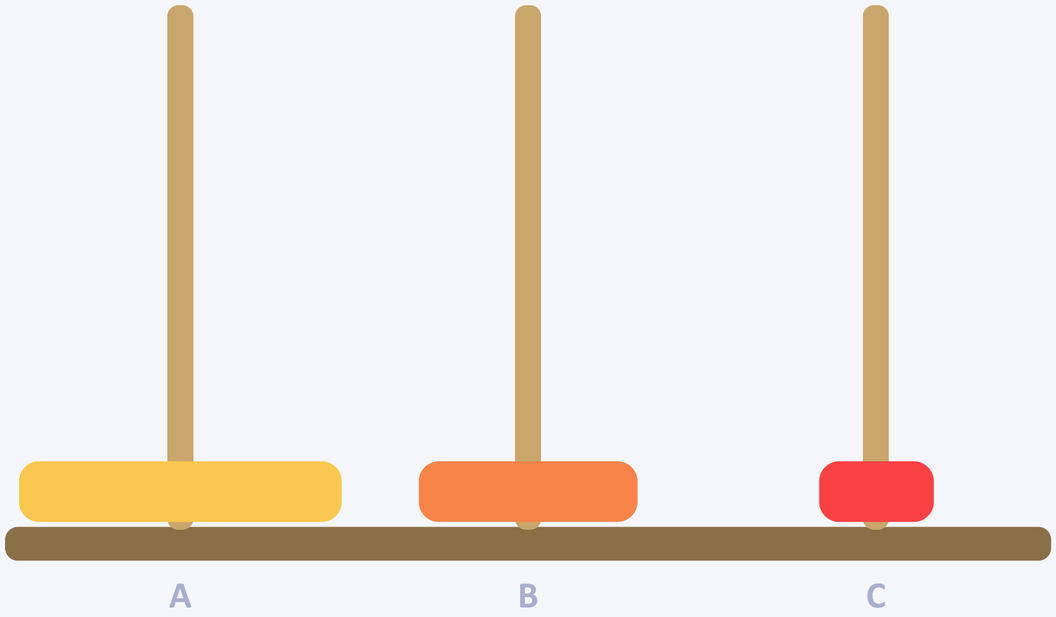


1 **A→C**

El disco 1 se mueve en los pasos 1, 3, 5 y 7, siempre en el ciclo A → C → B → A.

Ejemplo con 3 discos

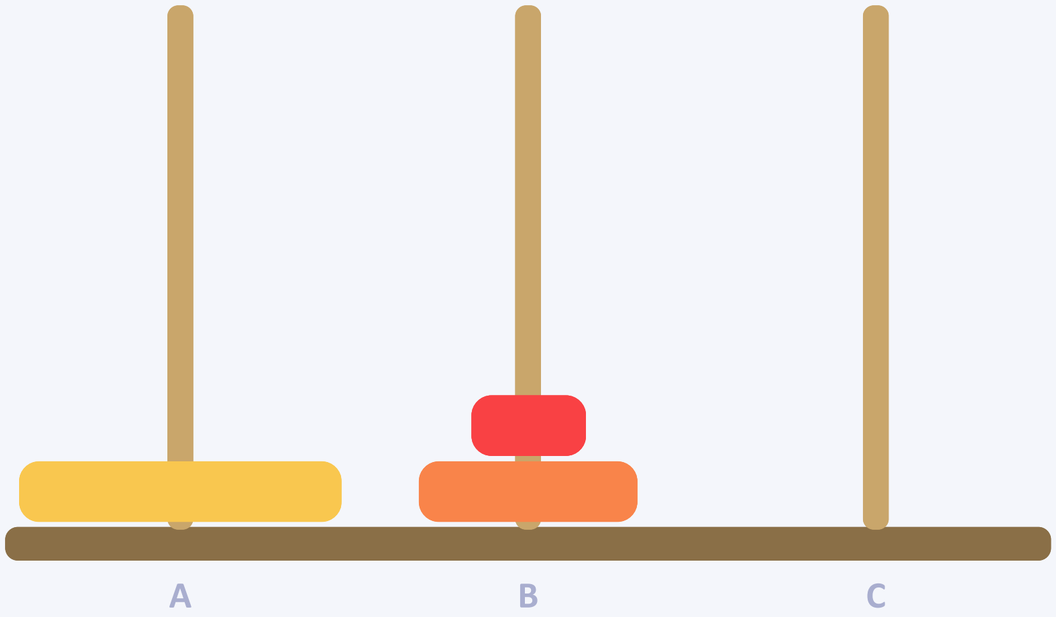
Los 7 movimientos de A (origen) a C (destino), con B como auxiliar



2 **A→B**

Ejemplo con 3 discos

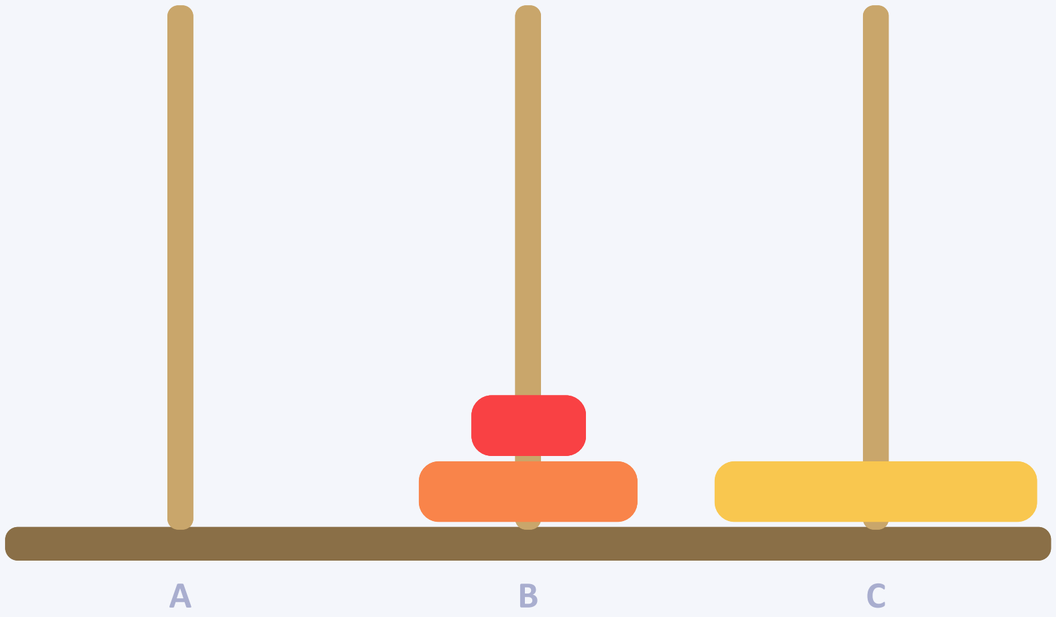
Los 7 movimientos de A (origen) a C (destino), con B como auxiliar



3 **C→B**

Ejemplo con 3 discos

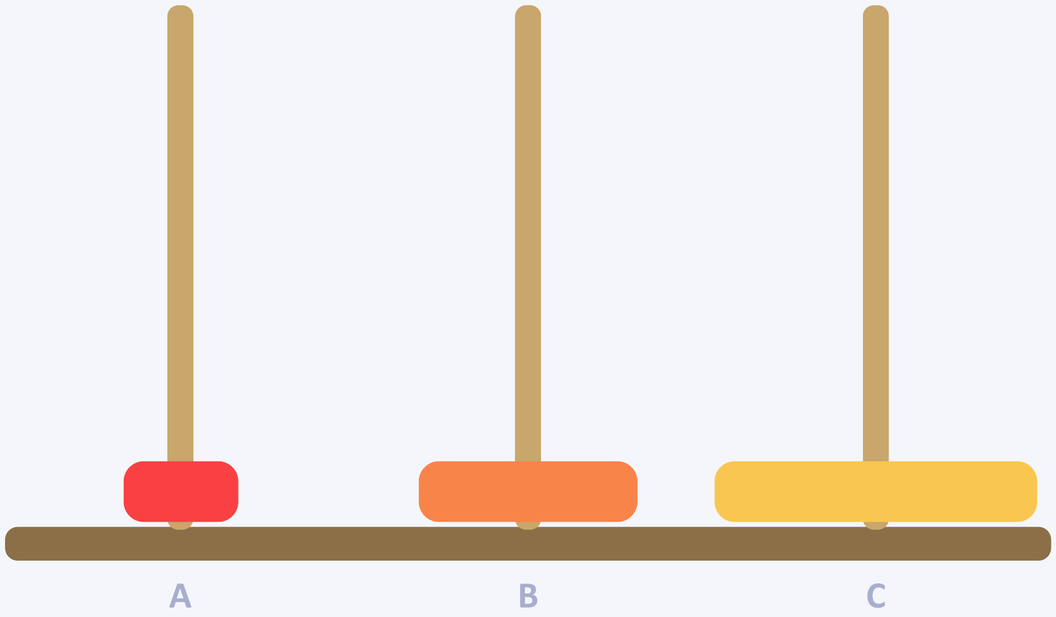
Los 7 movimientos de A (origen) a C (destino), con B como auxiliar



4 **A→C**

Ejemplo con 3 discos

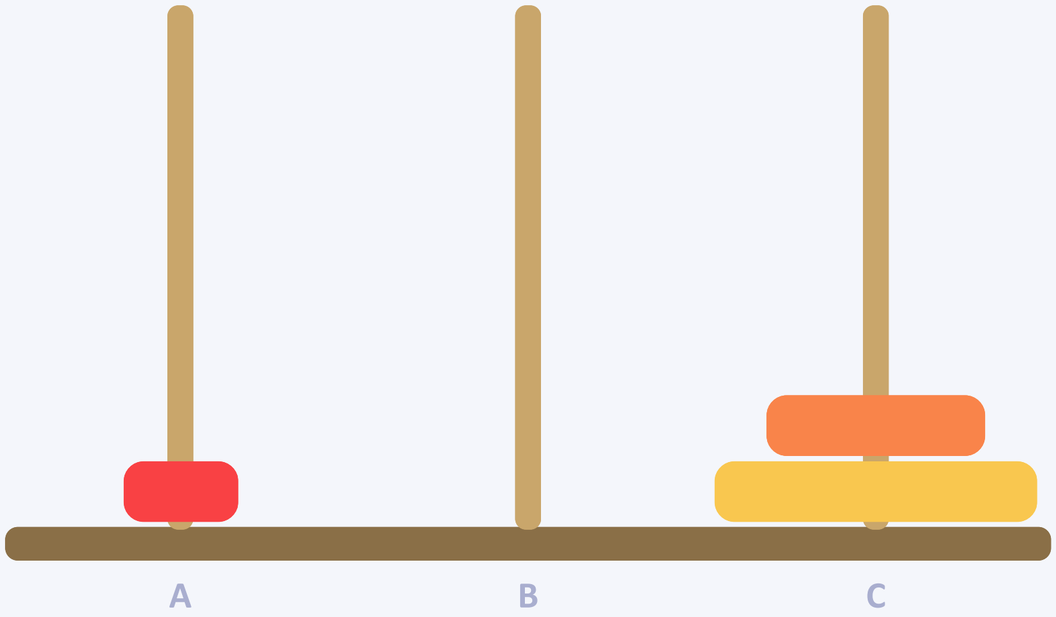
Los 7 movimientos de A (origen) a C (destino), con B como auxiliar



5 **B→A**

Ejemplo con 3 discos

Los 7 movimientos de A (origen) a C (destino), con B como auxiliar

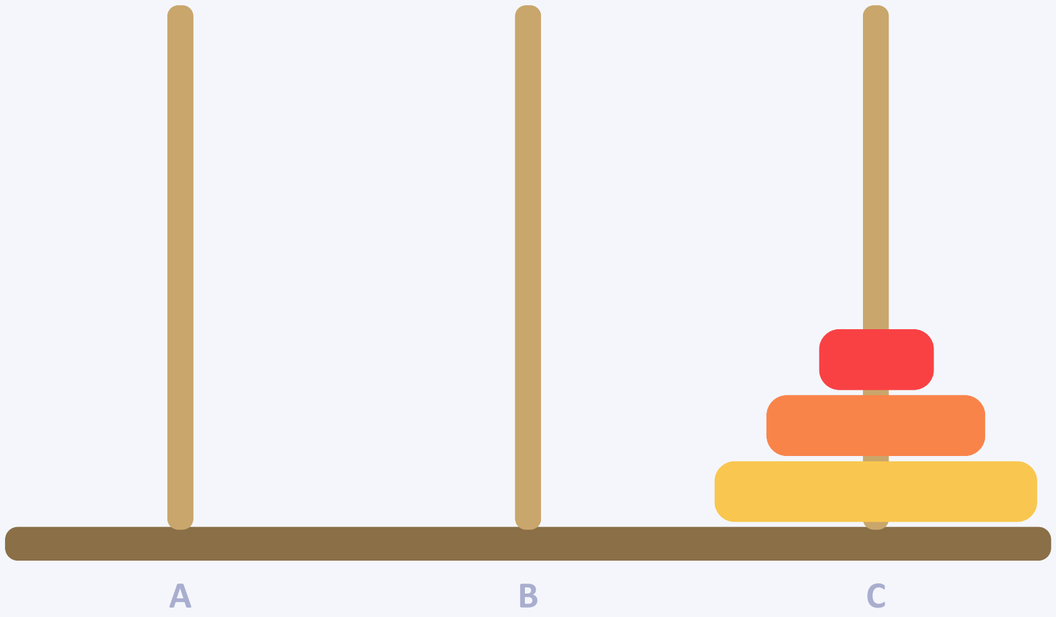


6 **B→C**

Ejemplo con 3 discos

Los 7 movimientos de A (origen) a C (destino), con B como auxiliar

Final



6 **A→C**

Iterativo sin stacks

Solo teoría: una regla simple que se repite en ciclo

1

Movimiento impar

Mueve el disco más pequeño a la siguiente torre, siempre en la misma dirección cíclica.

2

Movimiento par

Haz el único movimiento legal que no involucre al disco más pequeño.

3

Repite hasta terminar

Al llegar a $2^n - 1$ movimientos, todos los discos están en la torre destino.

Dirección del disco más pequeño

A origen · B auxiliar · C destino

n impar

$$A \to C \to B \to A$$

n par

$$A \to B \to C \to A$$

$$2^n - 1$$

movimientos mínimos con n discos

Stacks del método iterativo

Interfaz IStack e implementación StaticStack (arreglo de tamaño fijo)

1

Push, Pop y Peek

Agregar, quitar y ver el disco de arriba.

2

Size, Empty y Full

Indican si la torre está vacía o llena antes de mover un disco.

3

Torre inicial

new StaticStack<int>(n, 1) crea la torre origen con los discos n… 1, el 1 arriba.

```
interface IStack<T>
{
    int Size { get; }
    bool Empty { get; }
    bool Full { get; }
    void Push(T e);
    T Pop();
    T Peek();
}
```

```
public StaticStack(int capacity, int h)
{
    data = new T[capacity];

    for (int i = capacity - 1; i >= 0; i--)
    {
        T papu = (T)(object)h;
        data[i] = papu;
        h++;
    }

    index = capacity - 1;
}
```

Iterativo con stacks

Código: un bloque de 3 discos y el disco grande se turnan

1

Tres stacks

uno, dos y tres guardan los discos; el 1 (más pequeño) queda arriba.

2

MoverTres

Mueve 3 discos en 7 Push/Pop y se comporta como un solo disco.

3

Turnos alternados

Impar: mueve el bloque. Par: mueve el disco grande comparando Peek().

4

Total de turnos

2^(n−2) − 1. Si n es par el bloque va a la torre 2; si es impar, a la 3. Requiere n ≥ 3.

```
static void MoverTres(origen, auxiliar, destino)
{
    destino.Push(origen.Pop());    // 1
    auxiliar.Push(origen.Pop());   // 2
    auxiliar.Push(destino.Pop()); // 3
    destino.Push(origen.Pop());    // 4
    origen.Push(auxiliar.Pop());   // 5
    destino.Push(auxiliar.Pop()); // 6
    destino.Push(origen.Pop());    // 7
}
```

```
else if (discos > 3)
{
    IStack<int> bloqueEn;
    if (discos % 2 == 0)
    {
        MoverTres(uno, tres, dos);
        bloqueEn = dos;
    }
    else
    {
        MoverTres(uno, dos, tres);
        bloqueEn = tres;
    }

    long totalPasos = (long)Math.Pow(2, discos - 2) - 1;
```

Hacia dónde va el bloque

Iterativo con stacks: la dirección depende de si n es par o impar

Turnos impares: bloque de 3

n par

dos → tres → uno → dos

n impar

tres → dos → uno → tres

Turnos pares: disco grande

1

Ignora el bloque

Solo se miran las dos torres donde no está el bloque de 3.

2

Si una está vacía

El disco se mueve desde la otra torre hacia la vacía.

3

Si ninguna está vacía

Con Peek() se compara y el disco menor va sobre el mayor.

Recursivo

Divide y vencerás: n discos se resuelve con la solucion de n − 1

1. **Mueve n − 1 discos de A a B**
   usando C como auxiliar

2. **Mueve el disco n de A a C**
   el más grande va directo al destino

3. **Mueve n − 1 discos de B a C**
   usando A como auxiliar

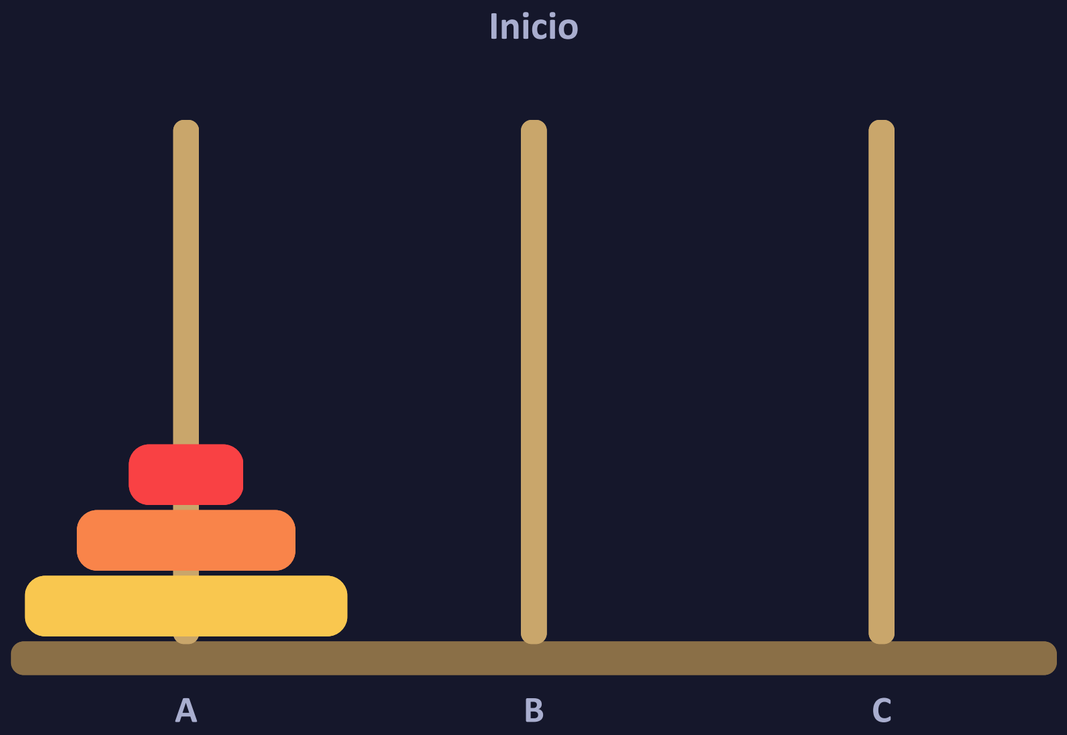
Caso base: n = 1 es un solo movimiento. De origen a destino.

```
static void Hanoi(
  int n,
  IStack<int> TO,
  IStack<int> TA,
  IStack<int> TD)
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (n-1, A, C, B)
        Hanoi(n - 1, TO, TD, TA);
        // Movemos disco N a destino.
        TD.Push(TO.Pop());
        // (n-1, B, A, C)
        Hanoi(n - 1, TA, TO, TD);
    }
}
```

$$T(n) = 2 \cdot T(n-1) + 1 \rightarrow 2^n - 1 \text{ movimientos}$$

Ejemplo con 3 discos

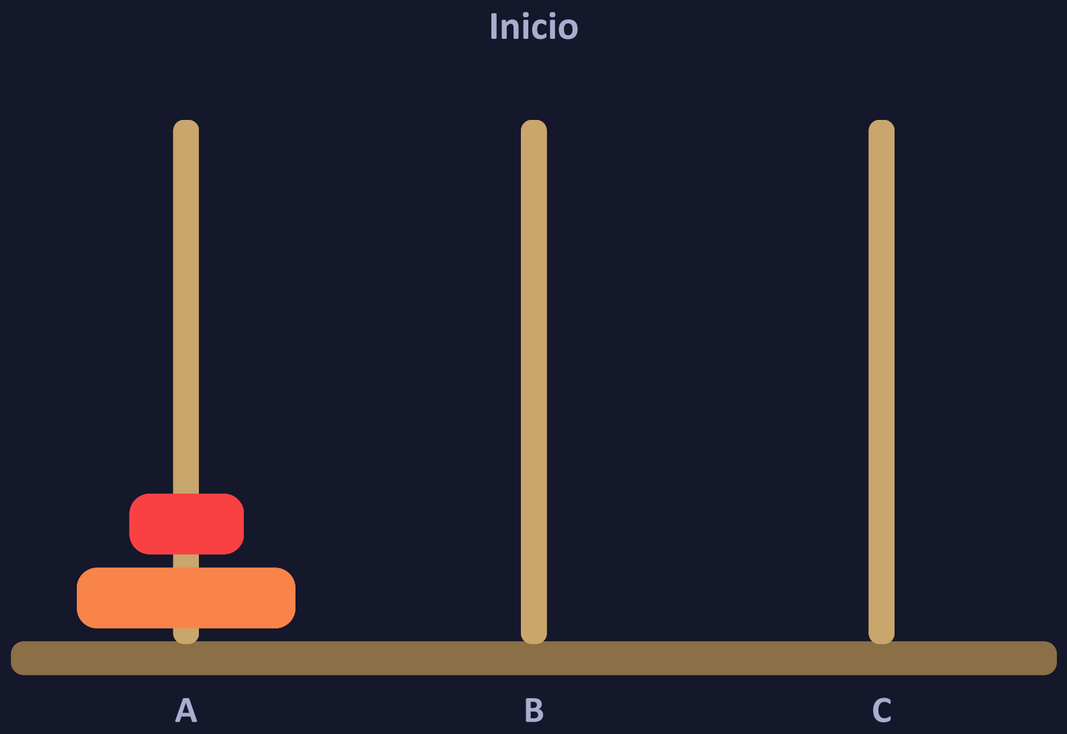
Los 7 movimientos de A (origen) a C (destino), con B como auxiliar



Hanoi(3, A, B, C);

```
static void Hanoi(
  int n, // 3
  IStack<int> TO, // A
  IStack<int> TA, // B
  IStack<int> TD) // C
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (2, A, C, B)
        Hanoi(n - 1, TO, TD, TA);
```

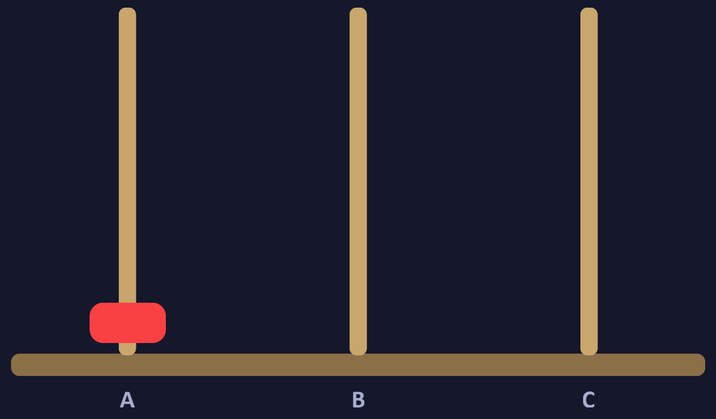
Ejemplo con 3 discos

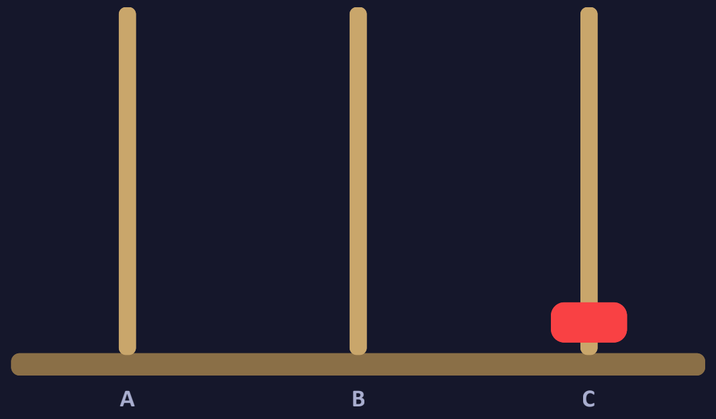


Hanoi(2, A, C, B);

```
static void Hanoi(
  int n, // 2
  IStack<int> TO, // A
  IStack<int> TA, // C
  IStack<int> TD) // B
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (1, A, B, C)
        Hanoi(n - 1, TO, TD, TA);
```

Ejemplo con 3 discos

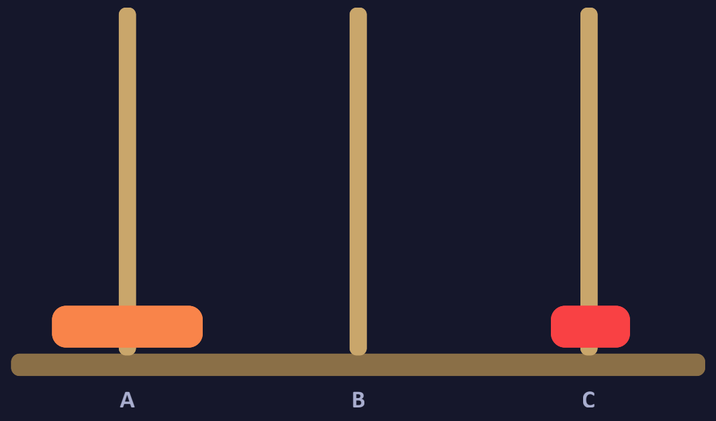


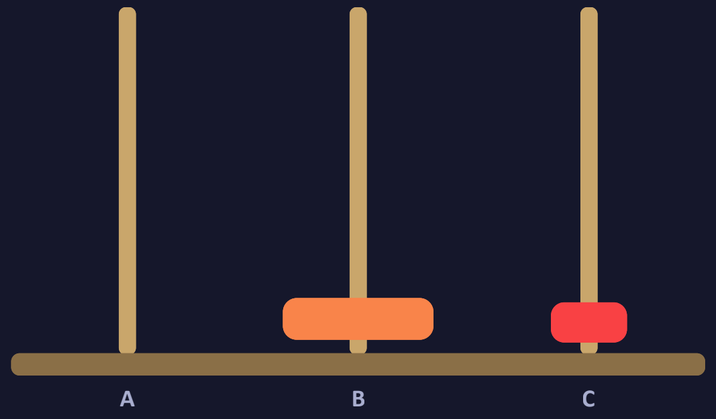


Hanoi(1, A, B, C);

```
static void Hanoi(
  int n, // 1
  IStack<int> TO, // A
  IStack<int> TA, // B
  IStack<int> TD) // C
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
```

Ejemplo con 3 discos

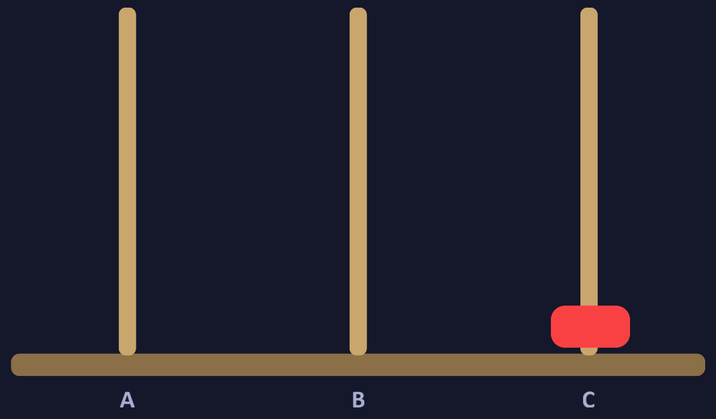


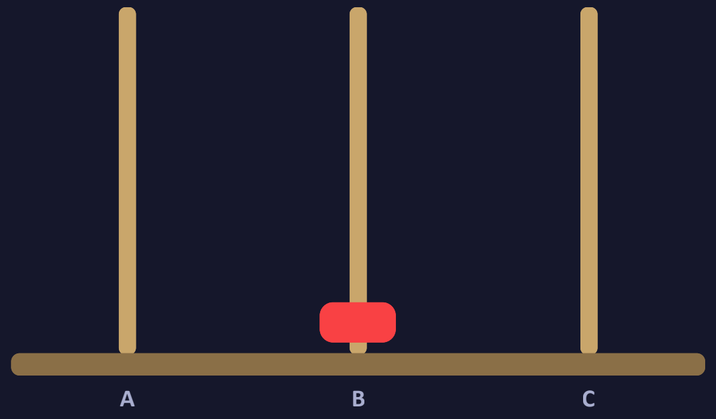


Hanoi(2, A, C, B);

```
static void Hanoi(
  int n, // 2
  IStack<int> TO, // A
  IStack<int> TA, // C
  IStack<int> TD) // B
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (1, A, B, C)
        Hanoi(n - 1, TO, TD, TA);
        // Movemos disco N a destino.
        TD.Push(TO.Pop());
        // (1, C, A, B)
        Hanoi(n - 1, TA, TO, TD);
```

Ejemplo con 3 discos

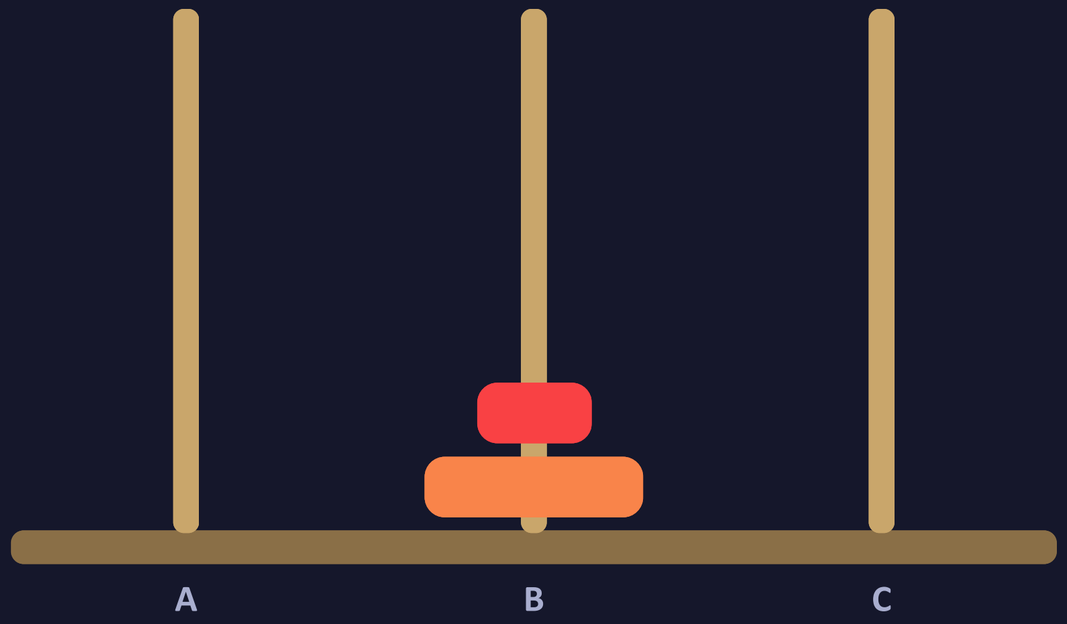




Hanoi(1, C, A, B);

```
static void Hanoi(
  int n, // 1
  IStack<int> TO, // C
  IStack<int> TA, // A
  IStack<int> TD) // B
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
```

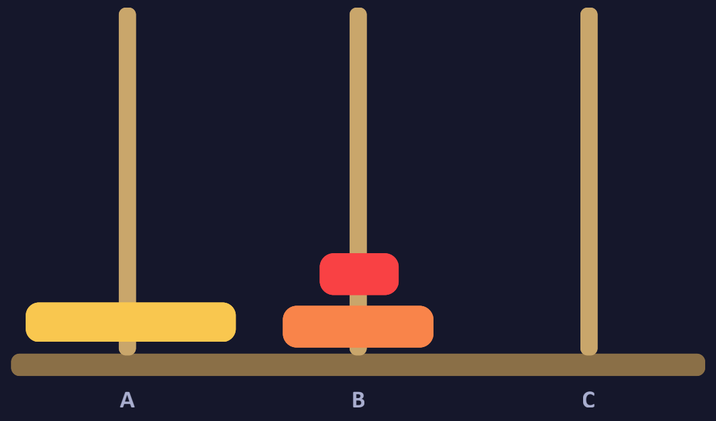
Ejemplo con 3 discos

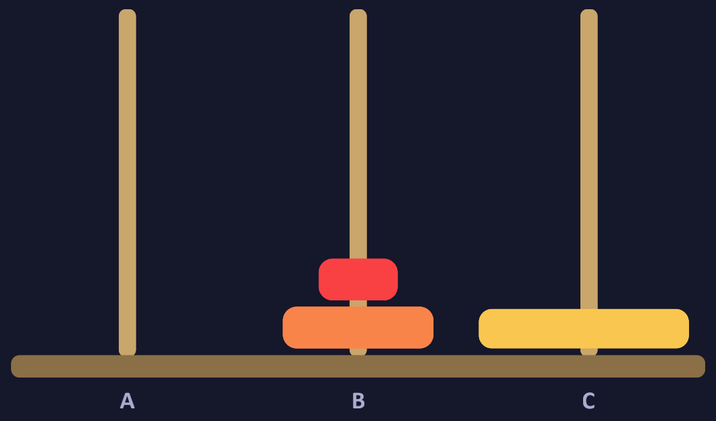


Hanoi(2, A, C, B);

```
static void Hanoi(
  int n, // 2
  IStack<int> TO, // A
  IStack<int> TA, // C
  IStack<int> TD) // B
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (1, A, B, C)
        Hanoi(n - 1, TO, TD, TA);
        // Movemos disco N a destino.
        TD.Push(TO.Pop());
        // (1, C, A, B)
        Hanoi(n - 1, TA, TO, TD);
    }
}   <----- Terminamos la iteracion.
```

Ejemplo con 3 discos

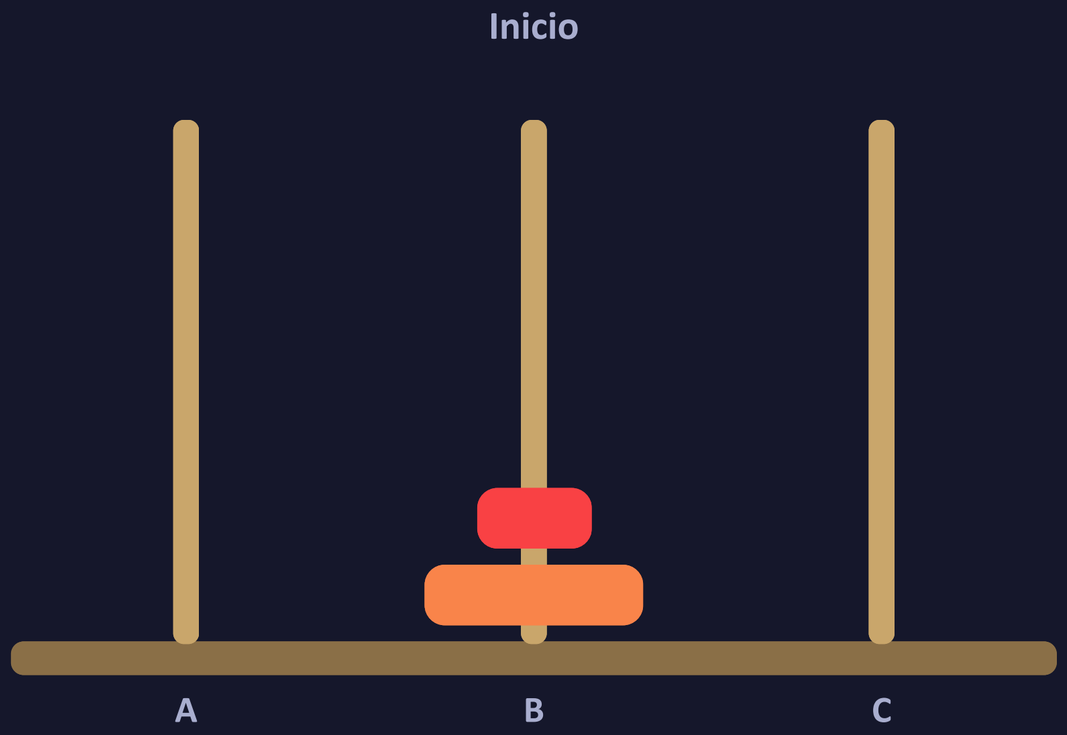




Hanoi(3, A, B, C);

```
static void Hanoi(
 int n, // 3
 IStack<int> TO, // A
 IStack<int> TA, // B
 IStack<int> TD) // C
{
    if (n == 1)
 {
 // Movemos disco N a destino
    TD.Push(TO.Pop());
    }
 else
 {
 // (2, A, C, B)
    Hanoi(n - 1, TO, TD, TA);
 // Movemos disco N a destino.
 TD.Push(TO.Pop());
 // (2, B, A, C)
 Hanoi(n - 1, TA, TO, TD);
```

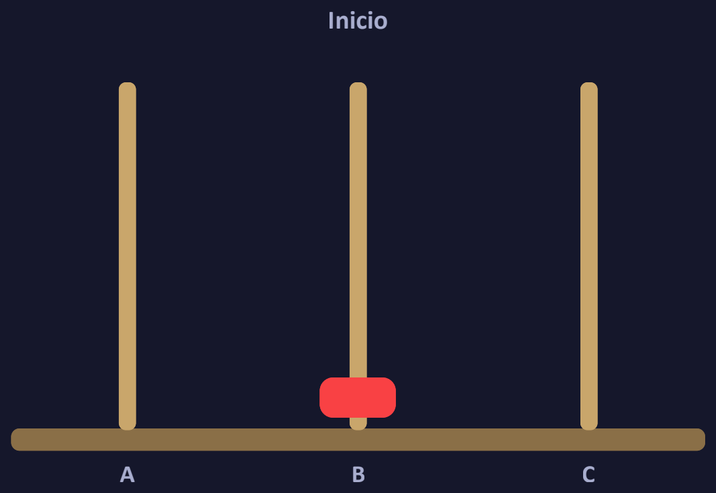
Ejemplo con 3 discos

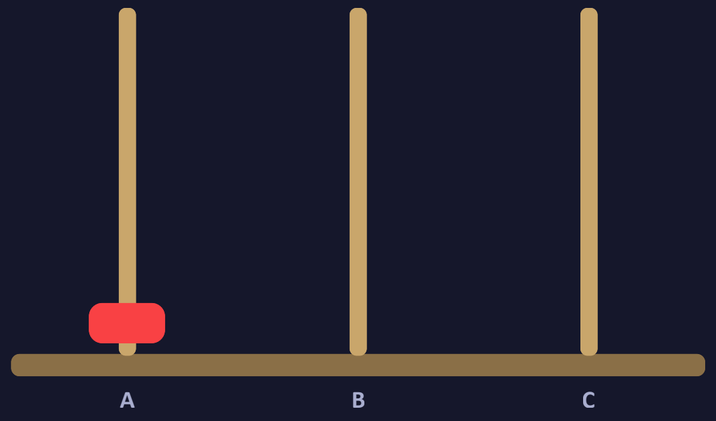


Hanoi(2, B, A, C);

```
static void Hanoi(
  int n, // 2
  IStack<int> TO, // B
  IStack<int> TA, // A
  IStack<int> TD) // C
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (1, B, C, A)
        Hanoi(n - 1, TO, TD, TA);
```

Ejemplo con 3 discos

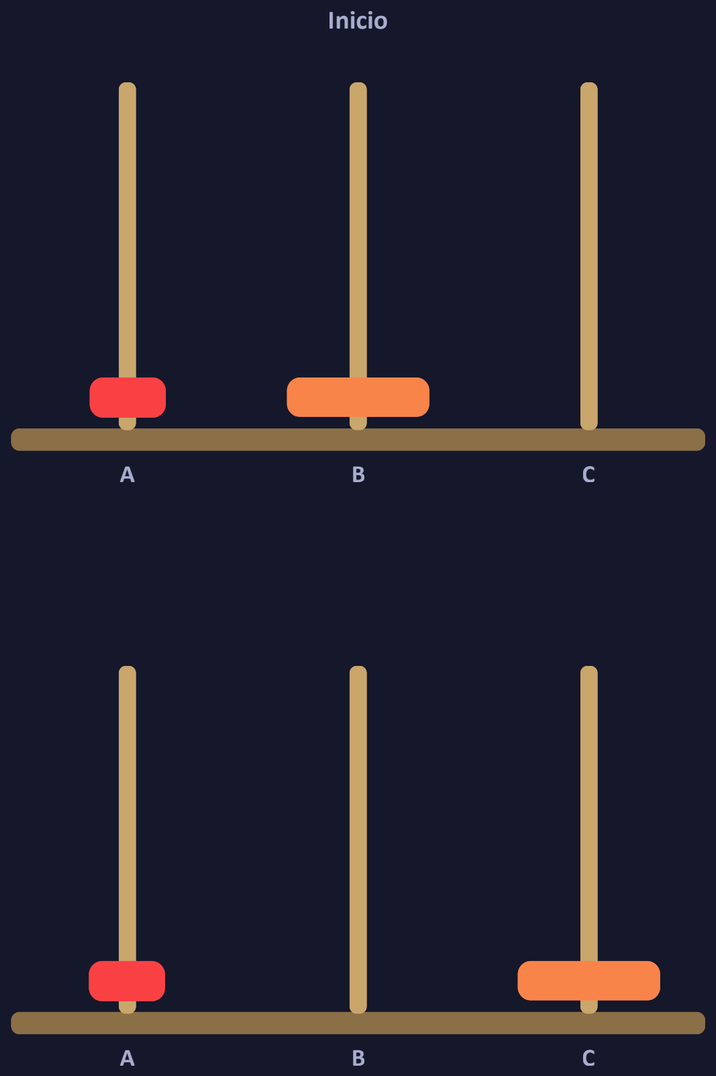




Hanoi(1, B, C, A);

```
static void Hanoi(
  int n, // 1
  IStack<int> TO, // B
  IStack<int> TA, // C
  IStack<int> TD) // A
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
```

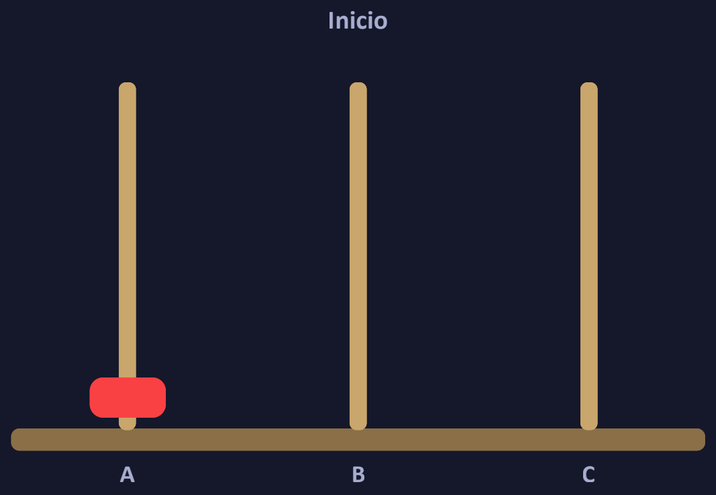
Ejemplo con 3 discos

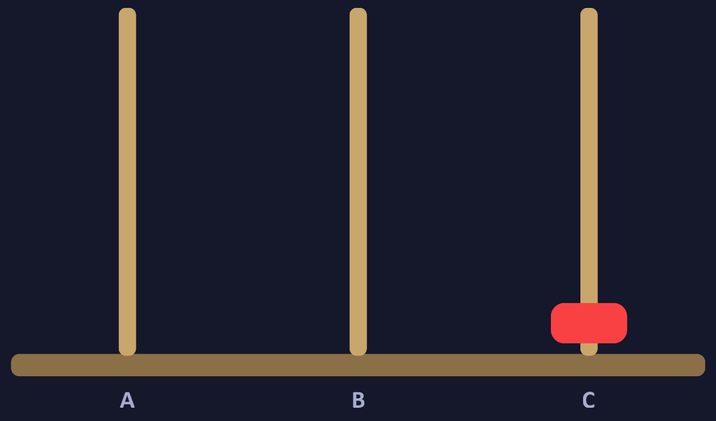


Hanoi(2, B, A, C);

```
static void Hanoi(
  int n, // 2
  IStack<int> TO, // B
  IStack<int> TA, // A
  IStack<int> TD) // C
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (1, B, C, A)
        Hanoi(n - 1, TO, TD, TA);
        // Movemos disco N a destino
        TD.Push(TO.Pop());
        // (1, A, B, C)
        Hanoi(n - 1, TA, TO, TD);
```

Ejemplo con 3 discos

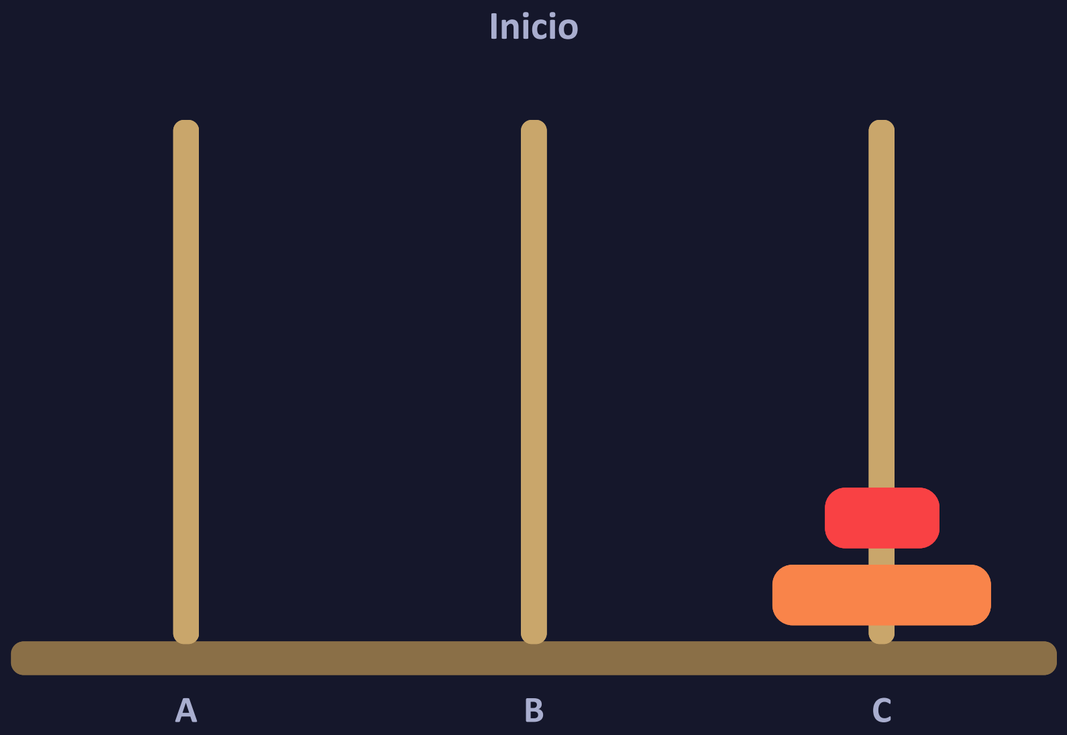




Hanoi(1, A, B, C);

```
static void Hanoi(
  int n, // 1
  IStack<int> TO, // A
  IStack<int> TA, // B
  IStack<int> TD) // C
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
```

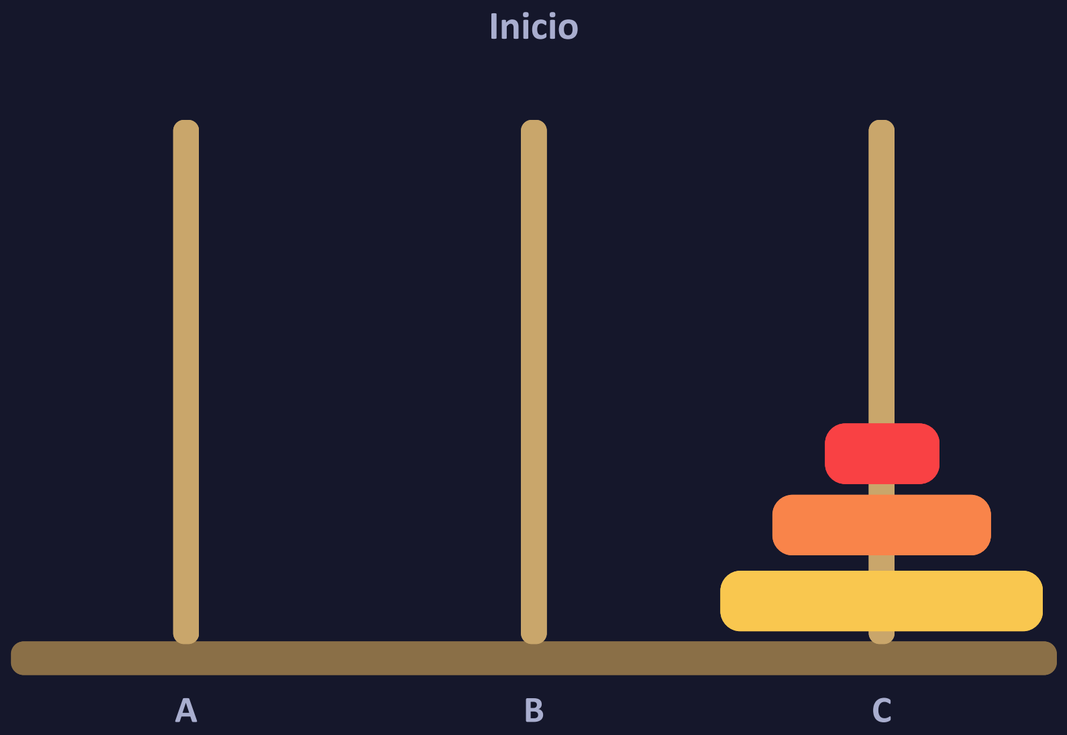
Ejemplo con 3 discos



Hanoi(2, A, C, B);

```
static void Hanoi(
  int n, // 2
  IStack<int> TO, // A
  IStack<int> TA, // C
  IStack<int> TD) // B
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (1, A, B, C)
        Hanoi(n - 1, TO, TD, TA);
        // Movemos disco N a destino.
        TD.Push(TO.Pop());
        // (1, C, A, B)
        Hanoi(n - 1, TA, TO, TD);
    }
}   <----- Terminamos la iteracion.
```

Ejemplo con 3 discos



Hanoi(3, A, B, C);

```
static void Hanoi(
  int n, // 3
  IStack<int> TO, // A
  IStack<int> TA, // B
  IStack<int> TD) // C
{
    if (n == 1)
    {
        // Movemos disco N a destino
        TD.Push(TO.Pop());
    }
    else
    {
        // (2, A, C, B)
        Hanoi(n - 1, TO, TD, TA);
        // Movemos disco N a destino.
        TD.Push(TO.Pop());
        // (2, B, A, C)
        Hanoi(n - 1, TA, TO, TD);
    }
}   <------------ Terminamos
```

Comparación

Tres formas de llegar al mismo resultado

Iterativo sin stacks

Idea

Disco pequeño en ciclo y el único movimiento legal restante.

Ventaja

Una sola regla, sin estructuras extra.

Desventaja

Hay que llevar el estado de las torres.

Iterativo con stacks

Idea

Bloque de 3 discos y disco grande, por turnos.

Ventaja

Sin recursión: el estado vive en los stacks.

Desventaja

Más código y casos (par/impar, n ≥ 3).

Recursivo

Idea

Mover n -1 a la torre auxiliar, mover n a la torre destino y mover n-1 a la torre destino

Ventaja

Código corto y fiel a la definición.

Desventaja

Usa la pila de llamadas: profundidad n.

Los tres hacen exactamente $2^n - 1$ movimientos.